SYSTEM DESIGN

Books & Step-by-Step Learning Roadmap

A practical 12-week plan to build fundamentals, reason about architectural trade-offs, and practice system design problems.

Read	Concepts before patterns; understand mechanisms and trade-offs.
Practice	Use the same design method for every problem.
Specialize	Build on distributed systems and big data experience.

Core principle: don't memorize hundreds of diagrams. Learn a compact set of fundamentals and apply them to new problems.

1. Recommended Books

1. Designing Data-Intensive Applications — Martin Kleppmann

Highest priority. Study databases, replication, partitioning, transactions, consistency, batch processing, and stream processing. Read slowly and focus on the mechanisms and trade-offs.

2. System Design Interview: An Insider's Guide, Volume 1 — Alex Xu

A practical guide to requirements, APIs, data models, architecture diagrams, and scaling decisions. Use it after learning the basic concepts.

3. System Design Interview: An Insider's Guide, Volume 2 — Alex Xu

Additional design examples and approaches. Identify reusable principles rather than memorizing the final diagrams.

4. Fundamentals of Software Architecture — Mark Richards and Neal Ford

Study architectural styles, modularity, coupling, cohesion, service boundaries, and architecture trade-offs. The second edition was published in 2025.

5. Understanding Distributed Systems — Roberto Vitillo

An optional, focused resource for reasoning about distributed components, failure handling, and reliability. Optional if DDIA already gives you the depth you need.

6. Designing Event-Driven Systems — Ben Stopford

A specialist resource for event streaming, Kafka-based architectures, event-driven systems, and data integration. Not a prerequisite for starting.

What to buy first: Start with DDIA. Add Fundamentals of Software Architecture for architectural reasoning, then Alex Xu Volume 1 for interview practice. You do not need to buy all six at once.

Useful publisher/author pages

- DDIA — official author page: https://dataintensive.net/
- Fundamentals of Software Architecture — O'Reilly: https://www.oreilly.com/library/view/fundamentals-of-software/9781098175504/
- System Design Interview — ByteByteGo: https://www.bytebytego.com/

2. A 12-Week Learning Roadmap

Weeks 1–2 — Fundamentals

Study: Networking, APIs, database indexes, transactions, caching, load balancing, and capacity estimation.

Deliverable: Draw a basic web application architecture and explain each component.

Weeks 3–5 — Distributed systems

Study: Selected DDIA topics: replication, partitioning, transactions, consistency, failures, and consensus.

Deliverable: Explain what happens when a database node fails or replicas disagree.

Weeks 6–7 — Architectural reasoning

Study: Read Fundamentals of Software Architecture: styles, modularity, coupling, cohesion, and trade-offs.

Deliverable: Justify monolith vs. microservices for a concrete application.

Weeks 8–9 — Common designs

Study: Use Alex Xu Volume 1 for URL shortener, rate limiter, news feed, chat, and notifications.

Deliverable: Design each from requirements to failure handling without copying the solution.

Weeks 10–11 — Big data systems

Study: Batch vs. streaming, Kafka, Spark, data lakes, partitioning, event processing, and replay.

Deliverable: Design an event-ingestion and analytics platform.

Week 12 — Consolidation

Study: Choose three unfamiliar problems and give yourself 45–60 minutes for each.

Deliverable: Explain your design aloud, defend trade-offs, and list gaps to revisit.

Suggested pace: around 5–7 hours per week. Treat the schedule as flexible; spend extra time on concepts that remain unclear.

3. A Repeatable Process for Every Design

1. Clarify requirements. Identify users, core operations, constraints, and what is out of scope.

2. Estimate scale. Estimate requests per second, storage growth, bandwidth, and read/write ratios.

3. Define APIs and data models. Identify entities, ownership, indexes, access patterns, and consistency needs.

4. Draw the simplest viable architecture. Start with clients, services, databases, and external dependencies.

5. Identify bottlenecks. Ask which component fails first as traffic or data grows.

6. Scale selectively. Add caching, queues, replicas, partitioning, or services only when justified.

7. Design for failures. Consider retries, duplicate messages, timeouts, failover, recovery, and degraded operation.

8. Explain trade-offs. State what you gain, what complexity you introduce, and why the decision fits the requirements.

4. The Four Questions for Every Concept

■ What problem does it solve?

■ How does it work?

■ What are its limitations or failure modes?

■ When would you choose it over an alternative?

Example — Redis: Understand why caching helps, how invalidation works, how cache stampedes occur, what stale data means, and when caching is not worth the complexity.

5. The Five Questions Behind Every Design

How does it scale? What grows first: traffic, data, CPU, memory, or network usage?

How does it handle failures? What if a server, database, broker, or network link fails?

How does it maintain data correctness? What consistency, ordering, transaction, or deduplication guarantees are required?

Where are its bottlenecks? Which component limits throughput or creates unacceptable latency?

What trade-offs justify the architecture? Why is this better for the requirements than a simpler alternative?

6. Final Advice

For backend, distributed systems, and big data preparation, emphasize distributed systems fundamentals, database internals, messaging, performance, and data-intensive architecture. Spend less time memorizing product names and more time understanding mechanisms and trade-offs.

Remember: depth of reasoning and the ability to defend architectural decisions are more valuable than memorizing dozens of diagrams.